

Support Vector Machines

from Scratch

Line-by-line theoretical commentary on `svm_from_scratch.py`

Lecture 02 — Finance Course

This document is a companion to `svm_from_scratch.py`. Every code block is reproduced verbatim, followed by a *line-by-line walkthrough*, the math behind it, and the practical pitfalls. Slides from Igor Halperin's *Supervised, Unsupervised and Reinforcement Learning in Finance* (NYU Tandon, 2017) are cross-referenced where relevant. Blue DERIVATION boxes carry the full algebra.

The slides are organised in three parts:

- **Part I** — primal SVR / SVC, ε -insensitive loss, soft margin C .
- **Part II** — KKT conditions, the dual problem, the support-vector expansion, convex optimisation.
- **Part III** — the kernel trick: features $\Phi(x)$ replaced by a kernel $k(x, x')$.

1. Mathematical recap

1.1 Primal soft-margin SVC (Part I)

The primal problem minimises the squared norm of the weight vector (equivalently, maximises the margin width $2/\|w\|$) plus a penalty C times the total slack:

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad (1)$$

$$\text{s.t.} \quad y_i(\langle w, x_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n. \quad (2)$$

1.2 Where does the margin width $2/\|w\|$ come from?

Derivation : geometric margin

Goal. Prove that the perpendicular distance between H_+ and H_- equals $2/\|w\|$, so that minimising $\|w\|^2$ maximises the margin width.

Step 1: identify the three relevant hyperplanes. The decision hyperplane H_0 is the zero level set of f :

$$H_0 = \{x \in \mathbb{R}^d : \langle w, x \rangle + b = 0\}.$$

The margin hyperplanes are the offsets at $f = \pm 1$:

$$H_+ = \{x : \langle w, x \rangle + b = +1\}, \quad H_- = \{x : \langle w, x \rangle + b = -1\}.$$

Why ± 1 specifically? If we scaled $(w, b) \rightarrow (cw, cb)$ for any $c > 0$, the same hyperplane is described, so the constraint $y_i(\langle w, x_i \rangle + b) \geq 1$ can always be enforced by rescaling. Fixing the offset at ± 1 pins down the scale of w .

Step 2: the normal direction. The gradient of f with respect to x is $\nabla_x f = w$, so w points normal to every level set of f — including H_0, H_+, H_- . The *unit* normal is therefore

$$\hat{n} = \frac{w}{\|w\|}, \quad \|\hat{n}\| = 1.$$

Step 3: pick a representative on H_+ and project. Choose any $x_+ \in H_+$. By the defining equation of H_+ :

$$\langle w, x_+ \rangle + b = +1 \Rightarrow \langle w, x_+ \rangle = 1 - b. \quad (*)$$

The signed distance from the origin to x_+ along $\hat{n}$ is the inner product

$$d_+ := \langle \hat{n}, x_+ \rangle = \left\langle \frac{w}{\|w\|}, x_+ \right\rangle = \frac{\langle w, x_+ \rangle}{\|w\|} \stackrel{(*)}{=} \frac{1 - b}{\|w\|}.$$

Step 4: same for H_- . For any $x_- \in H_-$, $\langle w, x_- \rangle = -1 - b$, so

$$d_- := \langle \hat{n}, x_- \rangle = \frac{-1 - b}{\|w\|}.$$

Step 5: subtract. The margin width is the perpendicular distance between H_+ and H_- . Because both points have been projected onto the same axis $\hat{n}$, this distance is just $d_+ - d_-$:

$$\text{margin width} = d_+ - d_- = \frac{1 - b}{\|w\|} - \frac{-1 - b}{\|w\|} = \frac{(1 - b) - (-1 - b)}{\|w\|} = \frac{2}{\|w\|}.$$

Note the b cancels — the margin width does not depend on where the boundary sits, only on $\|w\|$.

Step 6: from margin to objective. Maximising $2/\|w\|$ is equivalent to minimising $\|w\|$, which (for $w \neq 0$) is equivalent to minimising $\frac{1}{2}\|w\|^2$. The factor of $\frac{1}{2}$ and the square are chosen because

$$\nabla_w \left(\frac{1}{2} \|w\|^2 \right) = w,$$

a clean gradient that drops out of the Lagrangian analysis.

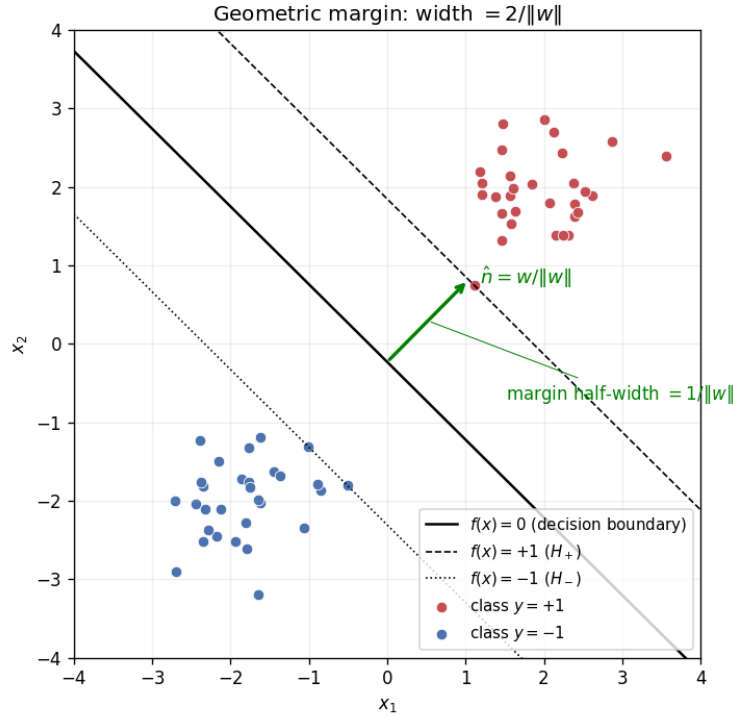


Figure 1. Two well-separated classes and the linear-SVM decision geometry. Solid line: decision boundary $f(x) = 0$. Dashed lines: the margin hyperplanes H_+ and H_- . Green arrow: the unit normal $\hat{n} = w / \|w\|$, with half-margin length $1 / \|w\|$. Full margin width is $2 / \|w\|$.

1.3 Lagrangian

Introducing multipliers $\alpha_i \geq 0$ on the margin constraints and $\mu_i \geq 0$ on the slack-non-negativity constraints:

$$L(w, b, \xi, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i (\langle w, x_i \rangle + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i. \quad (3)$$

1.4 Stationarity conditions

Derivation : stationarity, every step

Setup. Recall the Lagrangian

$$L(w, b, \xi, \alpha, \mu) = \underbrace{\frac{1}{2} \|w\|^2}_{(a)} + \underbrace{C \sum_i \xi_i}_{(b)} - \underbrace{\sum_i \alpha_i [y_i (\langle w, x_i \rangle + b) - 1 + \xi_i]}_{(c)} - \underbrace{\sum_i \mu_i \xi_i}_{(d)}.$$

The saddle-point conditions require $\partial L / \partial w$, $\partial L / \partial b$, $\partial L / \partial \xi_i$ to all vanish at the optimum.

Step 1: $\partial L / \partial w$. Differentiate each block:

$$\begin{aligned} \partial(a) / \partial w &= w & (\nabla_w \frac{1}{2} \|w\|^2 = w), \\ \partial(b) / \partial w &= 0 & (\xi_i \text{ has no } w), \\ \partial(c) / \partial w &= - \sum_i \alpha_i y_i x_i & (\langle w, x_i \rangle \text{ is linear in } w), \\ \partial(d) / \partial w &= 0 & (\xi_i \text{ has no } w). \end{aligned}$$

Sum and set to zero:

$$w - \sum_i \alpha_i y_i x_i = 0 \implies \boxed{w = \sum_i \alpha_i y_i x_i}.$$

This is the *primal recovery formula*: the optimal weight vector is a linear combination of training inputs, with coefficients $\alpha_i y_i$ given by the dual.

Step 2: $\partial L / \partial b$. Only block (c) has b :

$$\partial(c) / \partial b = \frac{\partial}{\partial b} \left(- \sum_i \alpha_i y_i b \right) = - \sum_i \alpha_i y_i.$$

Setting to zero gives the *equality constraint*

$$\boxed{\sum_i \alpha_i y_i = 0}.$$

Intuition. If $\sum_i \alpha_i y_i \neq 0$, we could shift b to lower L , so the saddle point cannot be there.

Step 3: $\partial L / \partial \xi_i$. For a fixed i , only three terms contain ξ_i :

$$\underbrace{C \xi_i}_{\text{from (b)}} - \underbrace{\alpha_i \xi_i}_{\text{from (c)}} - \underbrace{\mu_i \xi_i}_{\text{from (d)}} = \xi_i (C - \alpha_i - \mu_i).$$

Differentiating with respect to ξ_i and setting to zero:

$$C - \alpha_i - \mu_i = 0 \implies \mu_i = C - \alpha_i.$$

Now apply dual feasibility ($\alpha_i \geq 0$ and $\mu_i \geq 0$):

$$\mu_i \geq 0 \implies C - \alpha_i \geq 0 \implies \alpha_i \leq C.$$

Combined with $\alpha_i \geq 0$:

$$\boxed{0 \leq \alpha_i \leq C}.$$

The box constraint is the fingerprint of the soft margin. A hard-margin SVM (no slack) would have no upper bound on α_i .

1.5 Reducing the Lagrangian to a function of α only

Derivation : primal $\rightarrow$ dual, every cross-cancellation

Setup. We start from

$$L = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i (\langle w, x_i \rangle + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i.$$

Expand the bracketed term in the third sum:

$$L = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i y_i \langle w, x_i \rangle - b \sum_i \alpha_i y_i + \sum_i \alpha_i - \sum_i \alpha_i \xi_i - \sum_i \mu_i \xi_i.$$

Step 1: kill the ξ_i block. Collect the ξ_i terms:

$$C \sum_i \xi_i - \sum_i \alpha_i \xi_i - \sum_i \mu_i \xi_i = \sum_i \xi_i \underbrace{(C - \alpha_i - \mu_i)}_{=0 \text{ by stationarity}} = 0.$$

Slack vanishes from the dual.

Step 2: kill the b block.

$$-b \sum_i \alpha_i y_i = -b \cdot 0 = 0 \quad (\text{since } \sum_i \alpha_i y_i = 0).$$

Step 3: rewrite the $\langle w, x_i \rangle$ block using $w = \sum_j \alpha_j y_j x_j$.

$$- \sum_i \alpha_i y_i \langle w, x_i \rangle = - \left\langle \underbrace{\sum_i \alpha_i y_i x_i}_{=w}, w \right\rangle = - \langle w, w \rangle = - \|w\|^2.$$

Step 4: rewrite the $\|w\|^2$ block as a double sum. Using $w = \sum_i \alpha_i y_i x_i$ in both slots of $\langle \cdot, \cdot \rangle$:

$$\begin{aligned} \|w\|^2 &= \langle w, w \rangle \\ &= \left\langle \sum_i \alpha_i y_i x_i, \sum_j \alpha_j y_j x_j \right\rangle \\ &= \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle. \end{aligned}$$

Step 5: assemble. After Steps 1–3, only $\frac{1}{2} \|w\|^2 - \|w\|^2 + \sum_i \alpha_i$ survives:

$$L^*(\alpha) = \frac{1}{2} \|w\|^2 - \|w\|^2 + \sum_i \alpha_i = -\frac{1}{2} \|w\|^2 + \sum_i \alpha_i.$$

Substituting the double sum from Step 4:

$$L^*(\alpha) = \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle.$$

This is the dual objective. The constraints $\sum_i \alpha_i y_i = 0$ and $0 \leq \alpha_i \leq C$ come from the saddle-point feasibility of the multipliers we derived above.

So the dual problem is

$$\max_{\alpha} \quad \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(x_i, x_j) \quad (4)$$

$$\text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C. \quad (5)$$

1.6 The kernel trick (Part III)

Because (4) depends only on pairwise dot products, we can replace every $\langle x_i, x_j \rangle$ by a Mercer kernel $k(x, x') = \langle \Phi(x), \Phi(x') \rangle$ for some implicit feature map Φ . The script uses the linear kernel $k(x, x') = x^\top x'$ so $\Phi(x) = x$.

1.7 The support-vector expansion

Substituting $w = \sum_i \alpha_i y_i x_i$ into $f(x) = \langle w, x \rangle + b$:

$$f(x) = \sum_{i \in \text{SV}} \alpha_i y_i k(x_i, x) + b. \quad (6)$$

2. Imports

```
import cvxpy as cp
import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

Line by line.

► `import cvxpy as cp`

A domain-specific language for convex optimisation. Lets us write the SVM dual as a literal mathematical expression and hand it to a convex QP solver (CLARABEL by default), bypassing the SMO algorithm libsvm uses internally.

► `import numpy as np`

The numerical array library. Provides dot products, broadcasting, and matrix multiplications. The Gram matrix $K = XX^T$ is a single @ operation.

► `import pandas as pd`

Used only to assemble the synthetic dataset with labelled columns. Not strictly needed for the math; we convert to `X_dataframe.values` (NumPy) before any modelling.

► `from sklearn.metrics import roc_auc_score`

The rank-based area under the ROC curve. Independent of the class prior, so the right metric when defaults are imbalanced.

► `from sklearn.model_selection import train_test_split`

Sklearn's stratified random splitter. Used here on three parallel arrays simultaneously — it splits all of them with the same row indices.

► `from sklearn.preprocessing import StandardScaler`

Z-score scaler: subtracts the per-column mean and divides by the per-column standard deviation. Required because every kernel measures distances or dot products between feature vectors.

► `from sklearn.svm import SVC`

Sklearn's reference Support Vector Classifier (calling libsvm under the hood). We train it with identical hyperparameters and verify the from-scratch solution matches.

3. Reproducibility

```
RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
```

Line by line.

► `RANDOM_STATE = 42`

A single integer seeds every random draw downstream. Fixing it makes the from-scratch vs sklearn comparison deterministic — otherwise the QP would see two different training sets and we could not directly compare α , w , or b .

► `rng = np.random.default_rng(RANDOM_STATE)`

Returns a modern **Generator** object (PCG64 algorithm). Prefer this over the legacy `np.random.seed` + module-level functions; the modern generator has better statistical properties and is thread-safe.

4. Synthetic credit dataset

```
n_samples = 500

debt_to_income      = rng.beta(2, 5, n_samples) * 1.2
credit_utilization  = rng.beta(2, 4, n_samples)
missed_payments     = rng.poisson(0.6, n_samples)
savings_ratio       = rng.beta(2, 5, n_samples) * 0.5
income_volatility   = np.abs(rng.normal(0.15, 0.1, n_samples))
```

Line by line.

► `n_samples = 500`

Population size. Small enough that `cvxpy`'s interior-point solver finishes in a few seconds; large enough that the dual QP is well-conditioned.

► `debt_to_income = rng.beta(2, 5, n_samples) * 1.2`

Beta(2, 5) scaled to [0, 1.2]. The Beta distribution is the natural choice for a bounded ratio: density vanishes at the boundaries, peak interior. Mean $2/(2+5) = 0.286$ before scaling, 0.343 after.

► `credit_utilization = rng.beta(2, 4, n_samples)`

Beta(2, 4) in [0, 1]. Fraction of revolving-credit limit used. Mean $2/(2+4) = 0.333$ — realistic for an active credit card portfolio.

► `missed_payments = rng.poisson(0.6, n_samples)`

Poisson(0.6). Count of delinquencies in the last 12 months. Low mean $\lambda = 0.6$: about 55% of borrowers have 0 missed payments, 33% have 1, 10% have 2, and a thin tail beyond.

► `savings_ratio = rng.beta(2, 5, n_samples) * 0.5`

Beta(2, 5) scaled to [0, 0.5]. Savings as fraction of income. Skewed toward zero — realistic for retail borrowers.

► `income_volatility = np.abs(rng.normal(0.15, 0.1, n_samples))`

Half-normal centred at $\mu = 0.15$. Absolute value of a Gaussian is taken so the variable stays non-negative (volatility cannot be negative).

4.1 Logistic data-generating process

```
logit = (
    -4.0
    + 8.0 * debt_to_income
    + 5.0 * credit_utilization
    + 2.0 * missed_payments
    - 8.0 * savings_ratio
    + 5.0 * income_volatility
)
p_default = 1.0 / (1.0 + np.exp(-logit))
y_01      = (rng.uniform(size=n_samples) < p_default).astype(int)
y_signed  = 2 * y_01 - 1
```

Line by line.

► `logit = (-4.0 + 8.0 * debt_to_income + ...)`

Linear combination of risk factors: $\eta_i = \beta_0 + \beta_1 x_{i,1} + \dots + \beta_5 x_{i,5}$. Coefficients deliberately strong (8.0 on DTI vs 3.0 in `svm_minimal.py`) so the classes are cleanly linearly separable and the linear SVM has a non-degenerate optimum.

► `p_default = 1.0 / (1.0 + np.exp(-logit))`

Sigmoid: $p_i = \sigma(\eta_i) = 1/(1 + e^{-\eta_i}) \in (0, 1)$. Maps log-odds to probability.

► `y_01 = (rng.uniform(size=n_samples) < p_default).astype(int)`

Inverse-transform Bernoulli sampling: draw $U \sim \text{Uniform}(0, 1)$ once per borrower; $y_i = \mathbf{1}\{U_i < p_i\}$. By Derivation 4 below, $y_i \sim \text{Bernoulli}(p_i)$ exactly. The `.astype(int)` coerces the boolean array to $\{0, 1\}$.

► `y_signed = 2 * y_01 - 1`

Maps $\{0, 1\} \rightarrow \{-1, +1\}$. The slide math uses ± 1 everywhere (so that $y_i \cdot f(x_i)$ has the right sign convention in the margin constraints). We keep both representations for downstream use.

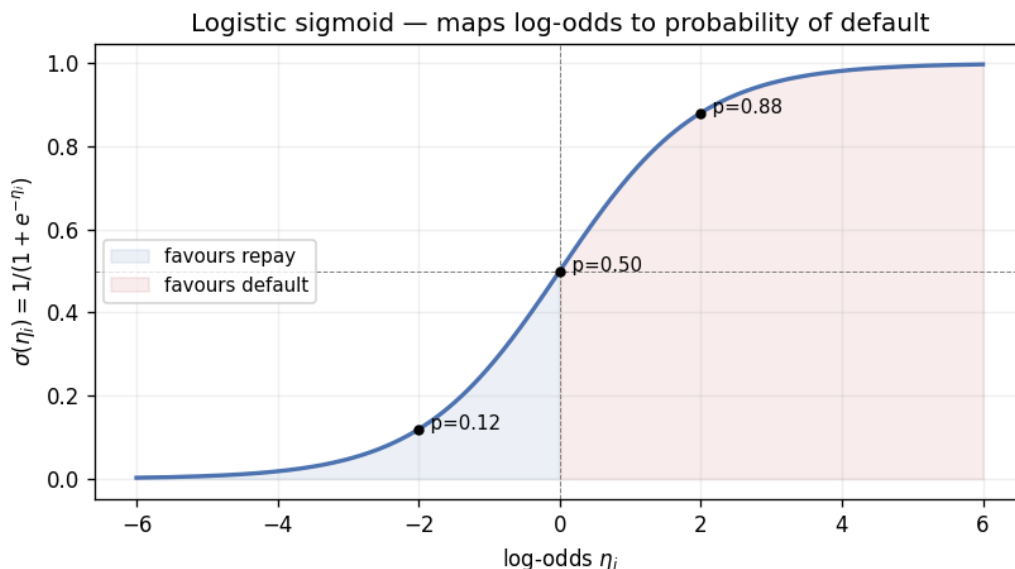


Figure 2. The logistic sigmoid maps the unbounded log-odds η_i to a probability $p_i \in (0, 1)$. $\eta = 0$ gives $p = 0.5$; large positive η saturates at $p = 1$; large negative η at $p = 0$. The shaded region (right) is where the model favours default; left, repay.

Derivation : inverse-transform Bernoulli sampling, every step

Setup. We have one fixed $p \in [0, 1]$ per borrower and a single $U \sim \text{Uniform}(0, 1)$. We want to construct y such that $y \sim \text{Bernoulli}(p)$.

Step 1: the Bernoulli CDF. For $Y \sim \text{Bernoulli}(p)$:

$$F_p(t) = \Pr(Y \leq t) = \begin{cases} 0 & t < 0, \\ 1 - p & 0 \leq t < 1, \\ 1 & t \geq 1. \end{cases}$$

The CDF jumps by $1 - p$ at $t = 0$ and by p at $t = 1$.

Step 2: the (generalised) inverse CDF. Define

$$F_p^{-1}(u) = \inf\{t : F_p(t) \geq u\} = \begin{cases} 0 & u \leq 1 - p, \\ 1 & u > 1 - p. \end{cases}$$

Step 3: inverse-transform theorem. A general result in probability: if F^{-1} is the inverse CDF of any distribution, and $U \sim \text{Uniform}(0, 1)$, then $F^{-1}(U)$ has CDF F . Applied here, $F_p^{-1}(U) \sim \text{Bernoulli}(p)$.

Step 4: rewrite $F_p^{-1}(U)$ as an indicator. $F_p^{-1}(U) = 1$ iff $U > 1 - p$, and 0 otherwise. So

$$F_p^{-1}(U) = \mathbf{1}\{U > 1 - p\} = \mathbf{1}\{1 - U < p\}.$$

Since $1 - U \sim \text{Uniform}(0, 1)$ (the uniform is symmetric on $[0, 1]$), we can rename $1 - U$ to U and get the equally valid formula

$$y := \mathbf{1}\{U < p\}.$$

Step 5: direct verification.

$$\Pr(y = 1) = \Pr(U < p) = \int_0^p 1 \, du = p.$$

So $y \sim \text{Bernoulli}(p)$ exactly. The expression `(rng.uniform(...) < p_default).astype(int)` implements this vectorised over all n borrowers simultaneously.

Pitfall

If the logit coefficients are too weak, the soft-margin SVM degenerates to $w \approx 0$ (predict-all-majority) and the comparison to sklearn becomes noise-dominated.

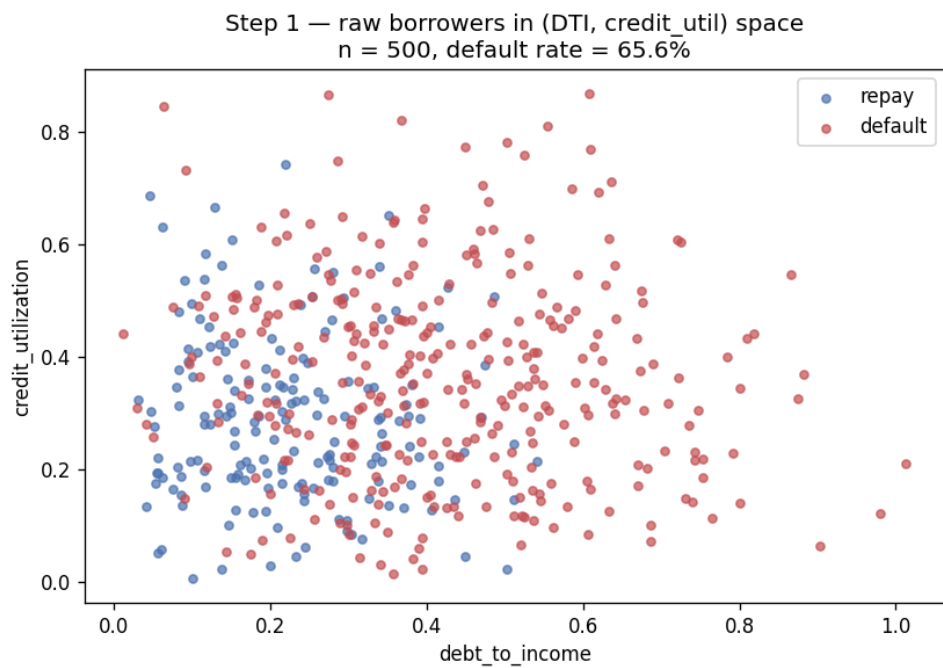


Figure 3. The synthetic credit book in the (DTI, credit-util) plane. Defaults concentrate in the upper right; this visible separation is what makes the linear SVM viable.

5. Stratified train/test split

```
(X_train, X_test,
 y_train_01, y_test_01,
 y_train, y_test) = train_test_split(
    X_dataframe.values, y_01, y_signed,
    test_size=0.25, stratify=y_01, random_state=RANDOM_STATE,
)
```

Line by line.

► `X_dataframe.values, y_01, y_signed`

Three parallel input arrays. `train_test_split` accepts any number of arrays as positional arguments and splits them in lockstep using the same row indices. Six outputs total: train/test halves for each input.

► `test_size=0.25`

25% of rows go to test, 75% to train. With $n = 500$ that yields exactly 375 train and 125 test rows (after stratified rounding).

► `stratify=y_01`

Class-balanced split. Sklearn samples within each class separately so that $\Pr(y = 1 \mid \text{train}) = \Pr(y = 1 \mid \text{test}) = \Pr(y = 1)$. The script verifies this in its output: **Default rate : train 65.60% test 65.60%.**

► `random_state=RANDOM_STATE`

Same seed used throughout the script. Makes the split (and therefore the entire downstream computation) deterministic and reproducible.

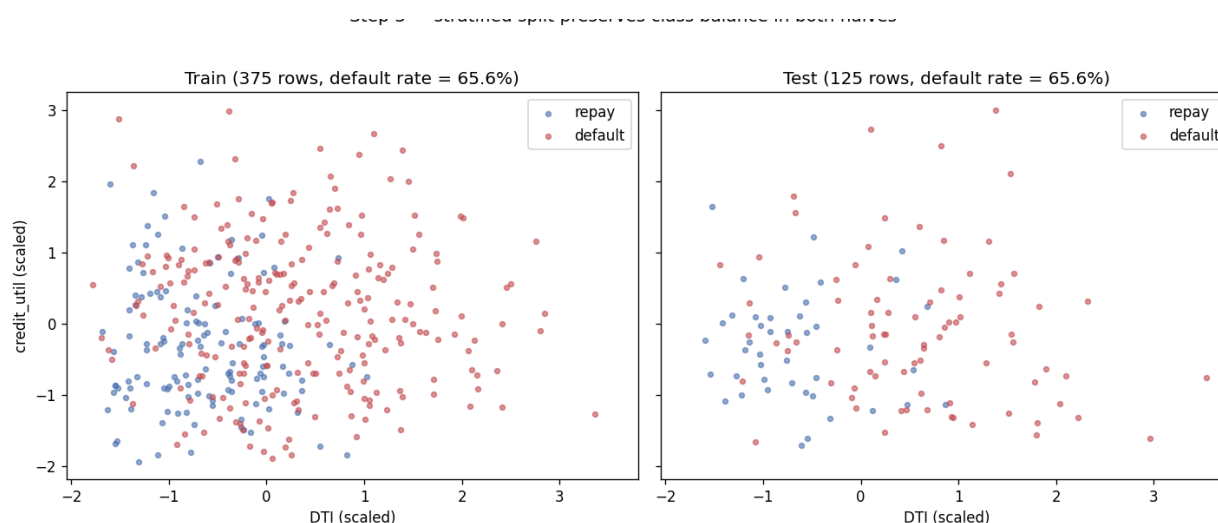


Figure 4. Stratified split. Both halves carry the same 65.6% default rate by construction.

Pitfall

Without stratification on small n , you can get unlucky splits like 70% / 50% defaults — biasing every metric.

6. Feature scaling

```
scaler      = StandardScaler().fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Line by line.

► `scaler = StandardScaler().fit(X_train)`

Fits the scaler on **X_train only**. Internally it computes $\mu_j = \text{mean}(X_{:,j})$ and $\sigma_j = \text{std}(X_{:,j})$ for each feature column j , storing them in `scaler.mean_` and `scaler.scale_`.

► `X_train_scaled = scaler.transform(X_train)`

Applies the stored (μ_j, σ_j) to every cell: $x'_{i,j} = (x_{i,j} - \mu_j) / \sigma_j$. After this every column has mean exactly 0 and std exactly 1.

► `X_test_scaled = scaler.transform(X_test)`

Applies the *same* (μ_j, σ_j) — the ones learned from training — to the test rows. The test set's own statistics are deliberately not used; otherwise we leak information from test back to train.

Derivation : why $\text{Var}(X') = 1$ after scaling, full proof

Setup. Let $X' \in \mathbb{R}^{n \times d}$ denote the scaled feature matrix. Treat its nd entries as one big sample. We compute $\text{Var}(X')$ in two stages.

Step 1: column means and variances after scaling. By construction of `StandardScaler`, for every column j ,

$$\bar{X}'_{:,j} = 0, \quad \text{Var}(X'_{:,j}) = 1.$$

Step 2: pooled mean and pooled variance. The overall mean of X' (treating all nd entries as one sample) is

$$\bar{X}' = \frac{1}{nd} \sum_{i,j} X'_{i,j} = \frac{1}{d} \sum_j \bar{X}'_{:,j} = \frac{1}{d} \sum_j 0 = 0.$$

The pooled variance is

$$\text{Var}(X') = \frac{1}{nd} \sum_{i,j} (X'_{i,j} - \bar{X}')^2 = \frac{1}{nd} \sum_{i,j} (X'_{i,j})^2 \quad (\text{since } \bar{X}' = 0).$$

Step 3: law of total variance applied to the column structure. Decompose the inner double sum by column:

$$\sum_{i,j} (X'_{i,j})^2 = \sum_j \sum_i (X'_{i,j})^2 = \sum_j n \cdot \text{Var}(X'_{:,j}) \quad (\text{using } \bar{X}'_{:,j} = 0).$$

Substitute back:

$$\text{Var}(X') = \frac{1}{nd} \sum_j n \cdot \text{Var}(X'_{:,j}) = \frac{1}{d} \sum_j \text{Var}(X'_{:,j}) = \frac{1}{d} \cdot d \cdot 1 = 1.$$

Conclusion. Pooled variance is exactly 1 (up to float precision). This is the property that lets sklearn's `gamma="scale"` default,

$$\gamma = \frac{1}{n_{\text{features}} \cdot \text{Var}(X)},$$

land at a sensible kernel bandwidth on every scaled dataset without manual tuning.

Step 2: Feature scaling: same shape, common units

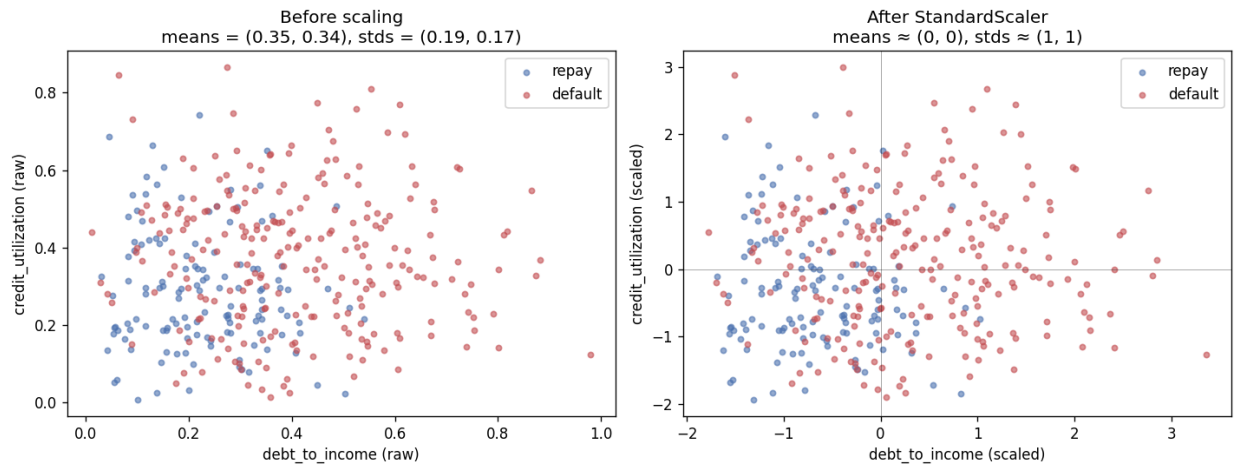


Figure 5. Before vs after `StandardScaler`: same shape, different location and scale.

Pitfall

Fitting the scaler on the full X before splitting leaks test-set statistics into training. Always fit on `X_train` only, then transform both.

7. Hyperparameter C

```
C_value = 10.0
```

Line by line.

► `C_value = 10.0`

The only hyperparameter the linear SVM has. Controls the trade-off between flatness (small $\|w\|^2$) and slack-violation tolerance. Small $C \rightarrow$ wide margin, simple model; large $C \rightarrow$ narrow margin, complex model. In the dual, C appears as the upper box bound $0 \leq \alpha_i \leq C$. Value 10 chosen because $C = 1$ left the optimum at $w \approx 0$ (predict-all-majority) on this dataset.

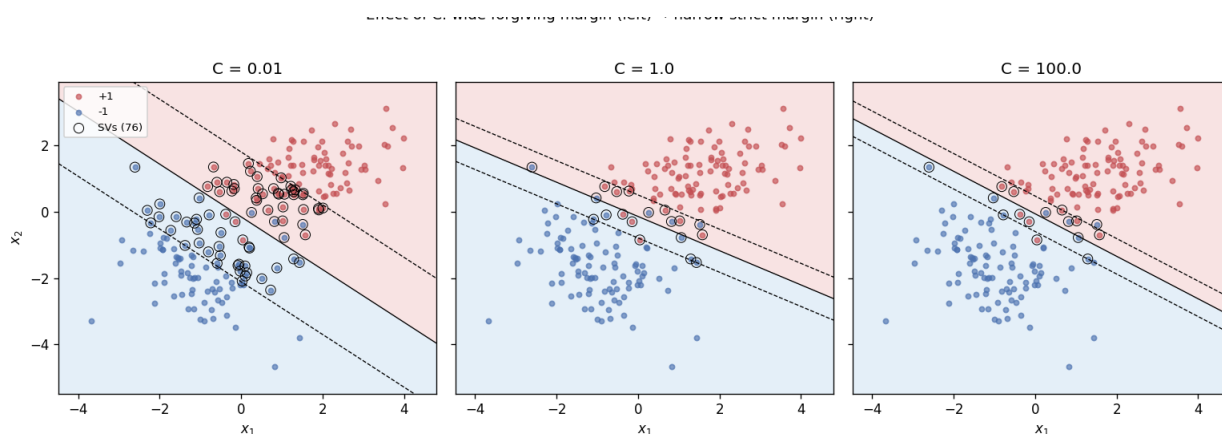


Figure 6. Effect of C on a partially overlapping two-class dataset. Left ($C = 0.01$): wide forgiving margin, many support vectors, almost no boundary tilt — high bias. Middle ($C = 1$): balanced solution. Right ($C = 100$): narrow strict margin, few SVs, boundary aggressively tilted to chase each violation — high variance.

8. Kernel (Gram) matrix

```
K_train = X_train_scaled @ X_train_scaled.T
```

Line by line.

► `K_train = X_train_scaled @ X_train_scaled.T`

For the linear kernel, the Gram matrix is literally one matrix multiplication: $K_{ij} = \langle x_i, x_j \rangle = (XX^\top)_{ij}$. Shape $(n_{\text{train}}, n_{\text{train}}) = (375, 375)$. PSD by construction (every Gram matrix is). Swap this one line for `rbf_kernel(X, X, gamma=g)` and you have an RBF SVM with everything below unchanged — the kernel trick in action.

Derivation : PSD-ness of any Gram matrix, every step

Definition recap. A symmetric matrix $K \in \mathbb{R}^{n \times n}$ is *positive semi-definite* (written $K \succeq 0$)

if

$$v^\top K v \geq 0 \quad \text{for every } v \in \mathbb{R}^n.$$

Step 1: write out $v^\top K v$ using $K_{ij} = \langle x_i, x_j \rangle$.

$$v^\top K v = \sum_{i=1}^n \sum_{j=1}^n v_i K_{ij} v_j = \sum_{i,j} v_i v_j \langle x_i, x_j \rangle.$$

Step 2: pull the scalars inside the inner product. Both arguments of $\langle \cdot, \cdot \rangle$ are linear, so $v_i v_j \langle x_i, x_j \rangle = \langle v_i x_i, v_j x_j \rangle$. Summing over i, j and using bilinearity again:

$$\sum_{i,j} \langle v_i x_i, v_j x_j \rangle = \left\langle \sum_i v_i x_i, \sum_j v_j x_j \right\rangle = \left\| \sum_i v_i x_i \right\|^2.$$

Step 3: conclude. A squared norm is non-negative:

$$v^\top K v = \left\| \sum_i v_i x_i \right\|^2 \geq 0.$$

So $K \succeq 0$, i.e. K is positive semi-definite.

When is K strictly positive definite? The chain above shows $v^\top K v = 0$ iff $\sum_i v_i x_i = 0$. So $K \succ 0$ (positive definite) iff the $\{x_i\}$ are linearly independent. For our problem $n_{\text{train}} = 375 \gg d = 5$, so the rows of X cannot all be independent — K is rank ≤ 5 , hence only PSD, not PD. That is why we need `cp.psd_wrap`: cvxpy would otherwise demand strict positivity to certify convexity.

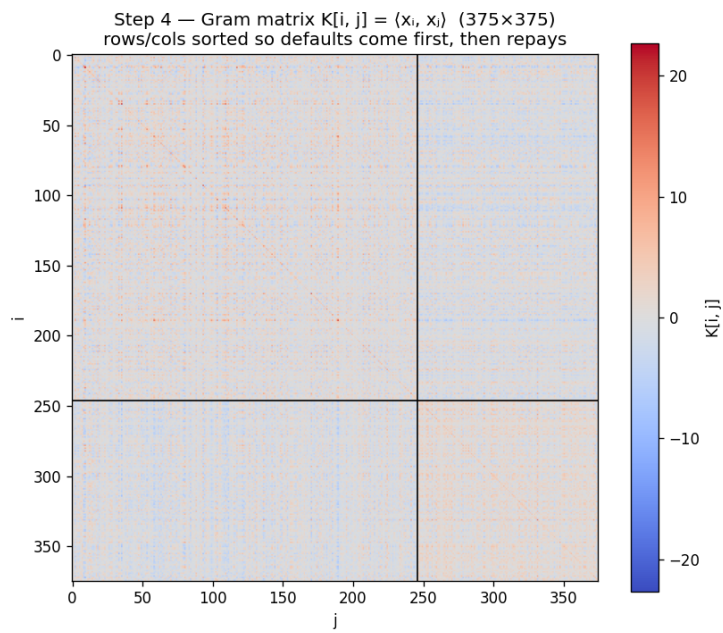


Figure 7. The Gram matrix with rows and columns sorted so defaults come first. The visible 2×2 block structure reveals within-class similarity vs cross-class similarity.

9. The Lagrangian dual as a convex QP

```
alpha    = cp.Variable(n_train, nonneg=True)
alpha_y  = cp.multiply(alpha, y_train)

dual_objective = cp.Maximize(
    cp.sum(alpha)
    - 0.5 * cp.quad_form(alpha_y, cp.psd_wrap(K_train))
)
dual_constraints = [
    alpha <= C_value,
    cp.sum(alpha_y) == 0,
]

dual_problem = cp.Problem(dual_objective, dual_constraints)
dual_problem.solve()
```

This is the heart of the script. The slide formula (4)–(5) is encoded literally in `cvxpy`.

Line by line.

► `alpha = cp.Variable(n_train, nonneg=True)`

Declares the dual decision variables α_i , one per training row. The keyword `nonneg=True` bakes the lower box constraint $\alpha_i \geq 0$ into the variable itself.

► `alpha_y = cp.multiply(alpha, y_train)`

Element-wise product $(\alpha_i y_i)$, the signed dual coefficient. After solving this equals sklearn's `dual_coef_[0]`. Used in the quadratic form and the equality constraint below.

► `cp.sum(alpha)`

Encodes the linear term $\sum_i \alpha_i$ of the dual objective. On its own, maximising this pushes every α_i to its upper bound C .

► `cp.quad_form(alpha_y, cp.psd_wrap(K_train))`

Encodes the quadratic form $(\alpha \circ y)^\top K (\alpha \circ y)$. By Derivation 7 this equals $\|w\|^2$, so the term in the objective is the dual rewriting of $\frac{1}{2} \|w\|^2$.

► `cp.psd_wrap(K_train)`

Asserts PSD-ness to `cvxpy`. Mathematically K is PSD (Derivation 6), but floating-point can leave a few eigenvalues at -10^{-16} instead of exactly zero. The wrap says “trust me, this is PSD” so `cvxpy` doesn't refuse the problem.

► `cp.Maximize( cp.sum(alpha) - 0.5 * cp.quad_form(...) )`

The full dual objective. Strong duality holds (Slater's condition: the primal is convex and strictly feasible), so the dual maximum equals the primal minimum.

► `alpha <= C_value`

The upper box. Element-wise vector inequality; combined with `nonneg=True` gives $0 \leq \alpha_i \leq C$, the signature of soft-margin SVM. A *hard*-margin SVM would have no upper bound.

► `cp.sum(alpha_y) == 0`

The equality constraint $\sum_i \alpha_i y_i = 0$ from $\partial L / \partial b = 0$. Forces a balanced α allocation between the two classes.

► `dual_problem = cp.Problem(dual_objective, dual_constraints)`

Wraps the objective and constraint list into a single `Problem` object. No solving yet — this is symbolic.

► `dual_problem.solve()`

This is where the QP actually runs. `cvxpy` canonicalises the problem to standard QP form $\min \frac{1}{2} x^\top P x + q^\top x$ s.t. $Gx \leq h, Ax = b$, then hands it to CLARABEL (primal-dual interior-point). After return: `dual_problem.status` = “optimal”, `dual_problem.value` = the optimal objective, `alpha.value` = the optimal α .

Derivation : $\|w\|^2 = (\alpha \circ y)^\top K (\alpha \circ y)$

Goal. Rewrite the primal regulariser $\|w\|^2$ purely in terms of the dual variables α and the kernel matrix K , with no explicit dependence on w .

Step 1: stationarity. From Derivation 2 (Step 1): $w = \sum_i \alpha_i y_i x_i$.

Step 2: expand the squared norm.

$$\begin{aligned} \|w\|^2 &= \langle w, w \rangle = \left\langle \sum_i \alpha_i y_i x_i, \sum_j \alpha_j y_j x_j \right\rangle \\ &\stackrel{\text{(bilinearity)}}{=} \sum_i \sum_j \langle \alpha_i y_i x_i, \alpha_j y_j x_j \rangle \\ &\stackrel{\text{(scalars out)}}{=} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle. \end{aligned}$$

Step 3: replace dot product with kernel value. By definition of the Gram matrix, $K_{ij} = \langle x_i, x_j \rangle$, so

$$\|w\|^2 = \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij}.$$

Step 4: matrix-vector form. Let $u = \alpha \circ y$ denote the element-wise product (so $u_i = \alpha_i y_i$). Then $\alpha_i \alpha_j y_i y_j = u_i u_j$, and the double sum collapses to a quadratic form:

$$\|w\|^2 = u^\top K u = (\alpha \circ y)^\top K (\alpha \circ y).$$

This is exactly what `cp.quad_form(alpha_y, K_train)` computes in the script. Multiplying by $\frac{1}{2}$ gives the dual’s quadratic regulariser.

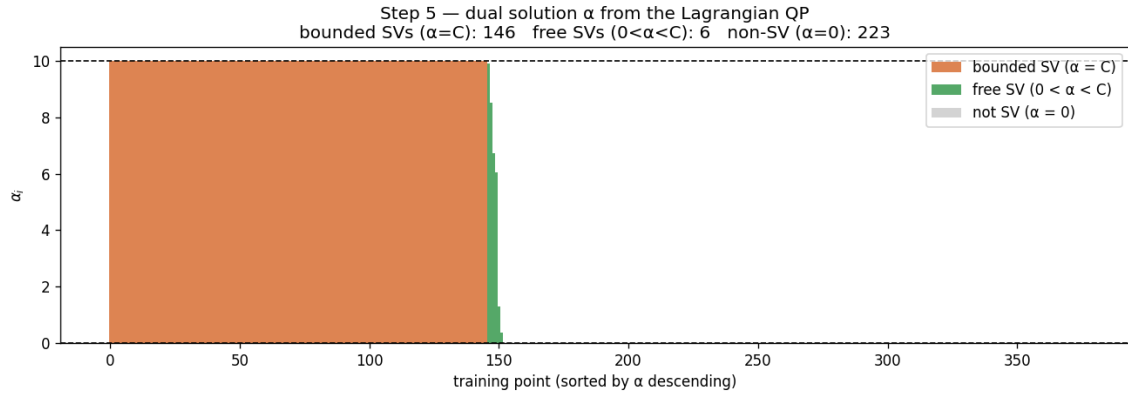


Figure 8. Dual solution α sorted descending. Bounded SVs ($\alpha_i = C$, orange) at the box; free SVs (green) interior; non-SVs (grey) at the flat tail.

10. Support-vector identification via KKT

```
TOL = 1e-5
sv_mask      = alpha_value > TOL
sv_indices   = np.where(sv_mask)[0]

free_mask    = (alpha_value > TOL) & (alpha_value < C_value - TOL)
free_indices = np.where(free_mask)[0]
bounded_indices = np.where(alpha_value >= C_value - TOL)[0]
```

Line by line.

► `TOL = 1e-5`

Numerical tolerance for “effectively zero” versus “effectively C ”. Necessary because the QP solver returns α values that satisfy the KKT conditions only up to its convergence tolerance, not exactly. 10^{-5} is comfortable for CLARABEL with default settings.

► `sv_mask = alpha_value > TOL`

Boolean array of length `n_train`. `True` at every training row whose α_i is positive — i.e. a support vector. By Derivation 8 (Case 1) the complement consists of points strictly outside the margin.

► `sv_indices = np.where(sv_mask)[0]`

Converts the boolean mask to the integer indices of the `True` entries. Used later to index into `support_vectors_` and `dual_coef_`.

► `free_mask = (alpha_value > TOL) & (alpha_value < C_value - TOL)`

Free SVs sit exactly on the margin: $0 < \alpha_i < C$, slack $\xi_i = 0$. By Case 2 of Derivation 8, the margin equation $y_i(\langle w, x_i \rangle + b) = 1$ holds with equality at these points — which is what makes them useful for recovering b .

► `free_indices = np.where(free_mask)[0]`

Integer indices of free SVs. Used in the next section to average over the free SVs when recovering b .

► `bounded_indices = np.where(alpha_value >= C_value - TOL)[0]`

Bounded SVs have $\alpha_i = C$ and slack $\xi_i > 0$ (Case 3). They sit inside the margin band or are outright misclassified.

Derivation : the three KKT cases, every implication

KKT system recap. At any optimal point the following hold for every i :

$$\begin{array}{ll} y_i(\langle w, x_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 & \text{(primal feasibility)} \\ \alpha_i \geq 0, \quad \mu_i \geq 0 & \text{(dual feasibility)} \\ \alpha_i [y_i(\langle w, x_i \rangle + b) - 1 + \xi_i] = 0 & \text{(c.s.1)} \\ \mu_i \xi_i = 0 & \text{(c.s.2)} \\ \mu_i = C - \alpha_i & \text{(stationarity in } \xi_i) \end{array}$$

We work case-by-case on the value of α_i relative to its box.

Case 1: $\alpha_i = 0$ (NOT a support vector).

- Stationarity gives $\mu_i = C - 0 = C > 0$.
- Then $\mu_i \neq 0$ combined with (c.s.2): $\xi_i = 0$.
- Primal feasibility with $\xi_i = 0$: $y_i(\langle w, x_i \rangle + b) \geq 1$.
- The point sits comfortably outside the margin band; it contributes *nothing* to the SV expansion.

Case 2: $0 < \alpha_i < C$ (FREE support vector).

- Stationarity gives $\mu_i = C - \alpha_i > 0$.
- $\mu_i \neq 0$ and (c.s.2): $\xi_i = 0$.
- $\alpha_i \neq 0$ and (c.s.1): $y_i(\langle w, x_i \rangle + b) - 1 + 0 = 0$, so $y_i(\langle w, x_i \rangle + b) = 1$.
- The point sits *exactly on the margin*, with slack zero.
- This is the case that lets us recover b (next derivation).

Case 3: $\alpha_i = C$ (BOUNDED support vector).

- Stationarity gives $\mu_i = C - C = 0$.
- $\mu_i = 0$, so (c.s.2) puts no constraint on ξ_i — it can be positive.
- $\alpha_i = C \neq 0$ and (c.s.1): $y_i(\langle w, x_i \rangle + b) = 1 - \xi_i \leq 1$.
- Sub-case 3a: $0 < \xi_i < 1$ — point inside the margin but on the correct side.
- Sub-case 3b: $\xi_i = 1$ — point exactly on the decision boundary.
- Sub-case 3c: $\xi_i > 1$ — point on the wrong side (misclassified).

Summary table.

α_i	geometry	slack ξ_i
0	strictly outside margin	0
$0 < \alpha_i < C$	exactly on margin	0
C	inside margin band or misclassified	> 0

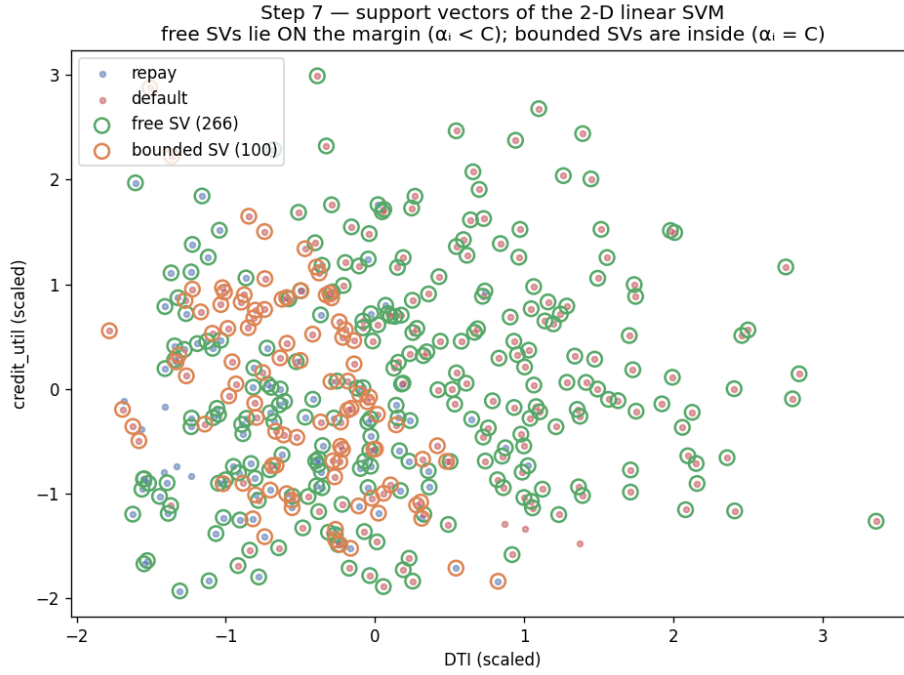


Figure 9. Training data with KKT classes highlighted. Free SVs (green) sit exactly on the margin; bounded SVs (orange) inside it; non-SVs are comfortably classified.

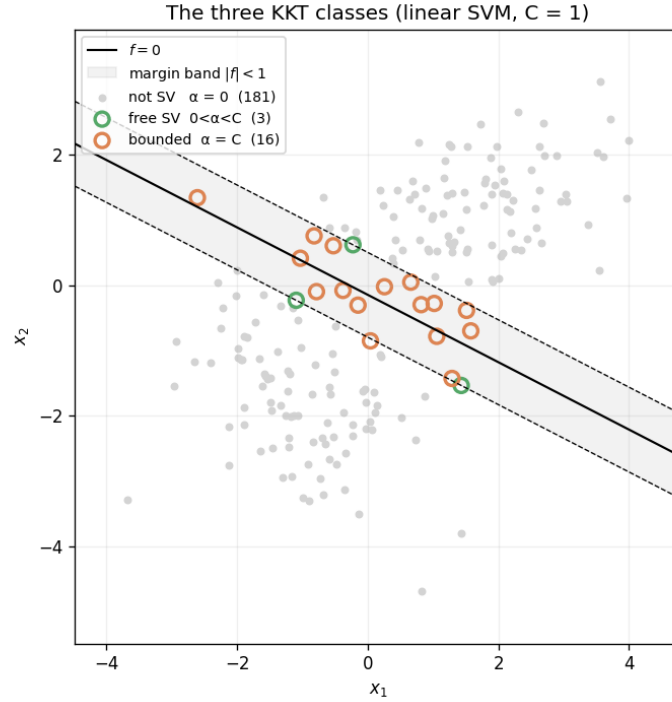


Figure 10. The same three KKT classes shown on a cleaner two-cluster dataset, with the margin band shaded in grey. **Not SVs** (light grey, $\alpha = 0$): comfortably outside the margin. **Free SVs** (green, $0 < \alpha < C$): sit exactly on $f = \pm 1$. **Bounded SVs** (orange, $\alpha = C$): inside the margin band or on the wrong side.

11. Recovering the primal weight vector w and the bias b

```
alpha_y_train = alpha_value * y_train
w_scratch     = alpha_y_train @ X_train_scaled

b_per_free_sv = y_train[free_indices] - X_train_scaled[free_indices] @ w_scratch
b_scratch     = float(np.mean(b_per_free_sv))
```

Line by line.

► `alpha_y_train = alpha_value * y_train`

Element-wise product $(\alpha_i y_i)$ as a NumPy array. Sklearn stores the same quantity in `svc.dual_coef_[0]` after fitting.

► `w_scratch = alpha_y_train @ X_train_scaled`

The stationarity formula $w = \sum_i \alpha_i y_i x_i$ in vectorised form. Shape: $(n_{\text{train}},) \cdot (n_{\text{train}}, d) \rightarrow (d,)$. Non-SVs contribute zero so summing over the full training set gives the same answer as summing only over support vectors.

► `b_per_free_sv = y_train[free_indices] - X_train_scaled[free_indices] @ w_scratch`

For every free SV, computes the bias estimate $b_i^{\text{est}} = y_i - \langle w, x_i \rangle$. Each free SV gives an estimate of b via Derivation 9.

► `b_scratch = float(np.mean(b_per_free_sv))`

Averages the per-free-SV estimates. Cancels solver-tolerance noise: any single SV's estimate would carry a numerical error proportional to the solver tolerance, but averaging over many cancels much of it.

Derivation : bias from a free SV, every step

Setup. By Case 2 of the KKT derivation, any free SV i satisfies the margin equation with equality:

$$y_i (\langle w, x_i \rangle + b) = 1. \quad (\dagger)$$

Step 1: multiply both sides by y_i .

$$y_i \cdot y_i (\langle w, x_i \rangle + b) = y_i \cdot 1.$$

Step 2: simplify y_i^2 . Since $y_i \in \{-1, +1\}$ we have $y_i^2 = 1$, so

$$\langle w, x_i \rangle + b = y_i.$$

Step 3: solve for b .

$$b = y_i - \langle w, x_i \rangle \quad (\text{any free SV } i).$$

Step 4: average for stability. In exact arithmetic, every free SV gives the same b . In floating point, each gives a slightly different value because α_i and w are themselves approximations. Averaging cancels these errors:

$$b = \frac{1}{|\mathcal{F}|} \sum_{i \in \mathcal{F}} (y_i - \langle w, x_i \rangle), \quad \mathcal{F} = \{i : 0 < \alpha_i < C\}.$$

The script implements this in two lines: `b_per_free_sv = y[free] - X[free] @ w` followed by `np.mean(b_per_free_sv)`.

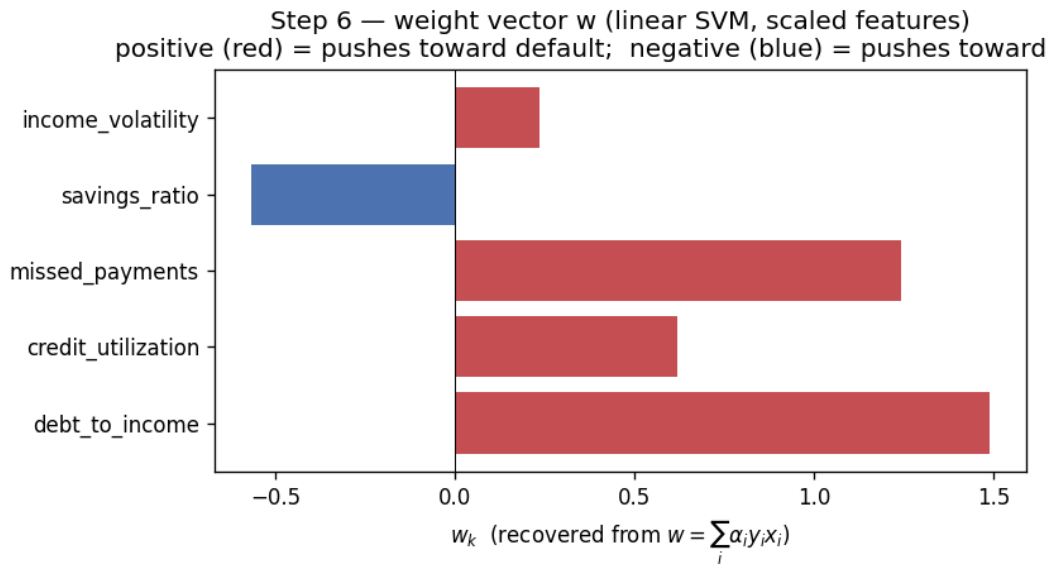


Figure 11. Recovered weight vector. Signs match financial intuition exactly: DTI, missed payments, credit utilisation, income volatility push toward default; savings push away.

Pitfall

If the QP solver returns no free SVs (only bounded ones, e.g. in pathological soft-margin regimes), this formula fails. `libsvm` uses a more elaborate KKT-balancing scheme to cover that corner case.

12. Decision function — primal and dual forms

```
# Primal form (only possible with linear kernel):
f_primal = X_test_scaled @ w_scratch + b_scratch

# Dual form (support-vector expansion):
K_test_sv = X_test_scaled @ X_train_scaled[sv_indices].T
f_dual    = K_test_sv @ alpha_y_train[sv_indices] + b_scratch
```

Line by line.

► `f_primal = X_test_scaled @ w_scratch + b_scratch`

Primal form: $f(x) = \langle w, x \rangle + b$. One $(n_{\text{test}}, d) \cdot (d, 1) = (n_{\text{test}}, 1)$ matmul plus a scalar broadcast. The fastest possible inference once w is known.

► `K_test_sv = X_test_scaled @ X_train_scaled[sv_indices].T`

Pairwise kernel evaluations between each test row and each support vector. Shape $(n_{\text{test}}, |\text{SV}|)$. This is the only piece of the dual form that scales with the number of SVs.

► `f_dual = K_test_sv @ alpha_y_train[sv_indices] + b_scratch`

Dual form / support-vector expansion: $f(x) = \sum_{i \in \text{SV}} \alpha_i y_i k(x_i, x) + b$. Algebraically identical to the primal form (Derivation 10); we compute it separately to verify the equivalence on the test set.

Derivation : primal $\equiv$ dual decision function

Claim. The two expressions

$$f_{\text{primal}}(x) := \langle w, x \rangle + b, \quad f_{\text{dual}}(x) := \sum_{i \in \text{SV}} \alpha_i y_i \langle x_i, x \rangle + b$$

are algebraically identical functions of x .

Step 1: substitute the stationarity formula for w .

$$f_{\text{primal}}(x) = \langle w, x \rangle + b = \left\langle \sum_{i=1}^n \alpha_i y_i x_i, x \right\rangle + b.$$

Step 2: pull the sum out of the inner product. Inner products are linear in each argument: $\langle \sum_i v_i, w \rangle = \sum_i \langle v_i, w \rangle$. Applied with $v_i = \alpha_i y_i x_i$:

$$\left\langle \sum_i \alpha_i y_i x_i, x \right\rangle = \sum_i \langle \alpha_i y_i x_i, x \rangle = \sum_i \alpha_i y_i \langle x_i, x \rangle.$$

Step 3: drop the non-support vectors. By KKT Case 1, non-SVs have $\alpha_i = 0$, so they contribute zero to the sum:

$$\sum_{i=1}^n \alpha_i y_i \langle x_i, x \rangle = \sum_{i \in \text{SV}} \alpha_i y_i \langle x_i, x \rangle.$$

Step 4: re-add b .

$$f_{\text{primal}}(x) = \sum_{i \in \text{SV}} \alpha_i y_i \langle x_i, x \rangle + b = f_{\text{dual}}(x).$$

The two functions are identical for every input.

Computational comparison.

	primal	dual
expression	$\langle w, x \rangle + b$	$\sum_i \alpha_i y_i k(x_i, x) + b$
cost per query	$O(d)$	$O(\text{SV} \cdot d)$
works for any kernel?	no (needs w)	yes (needs only k)

The numerical equivalence is verified by the script's $\max |f_{\text{primal}} - f_{\text{dual}}| = 4.88\text{e-}13$ (machine epsilon).

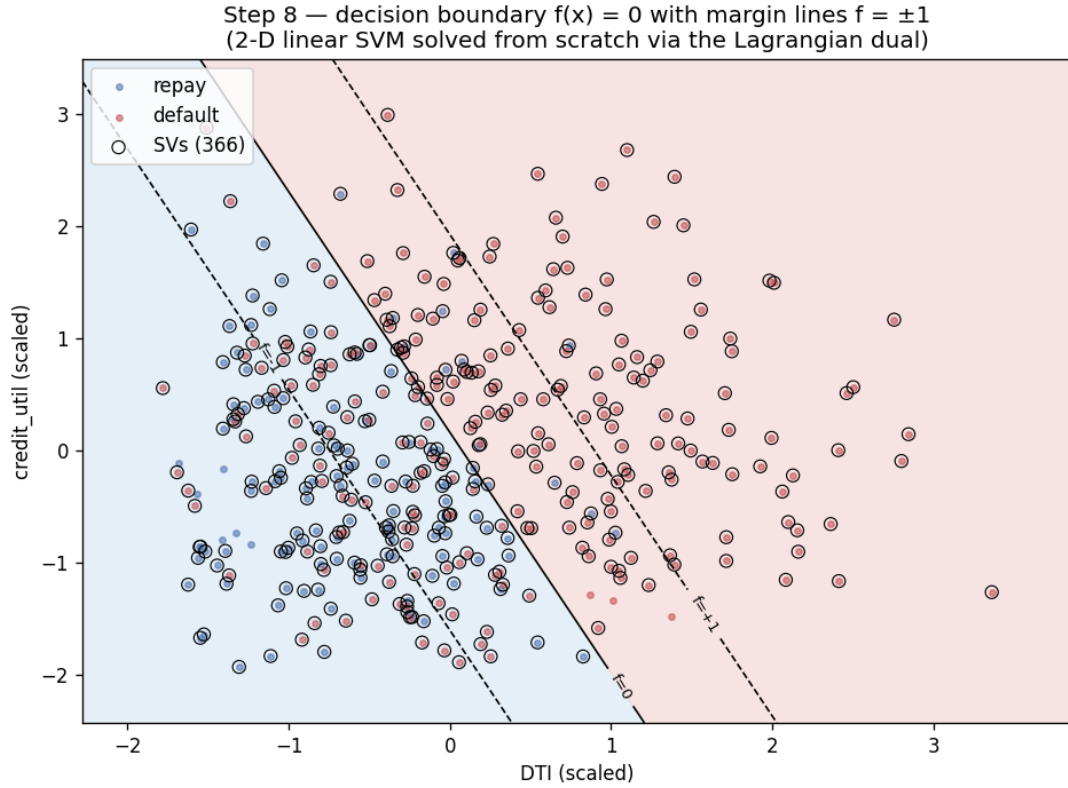


Figure 12. Decision boundary $f(x) = 0$ (solid) with margin lines $f = \pm 1$ (dashed). Open circles mark every support vector.

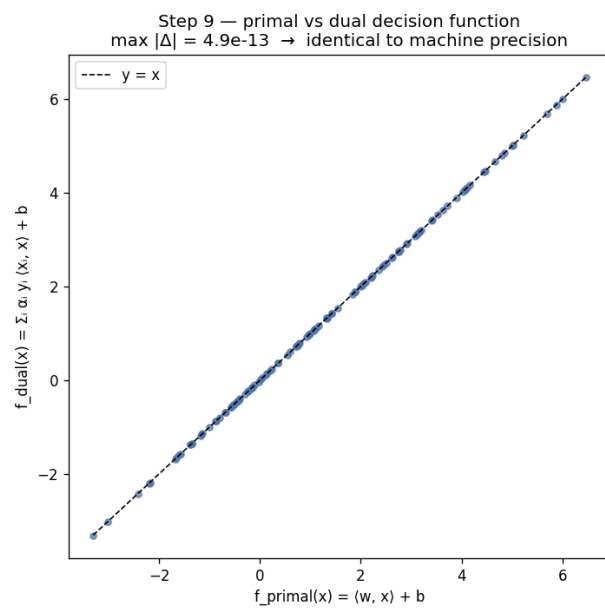


Figure 13. Test-set scatter of f_{primal} vs f_{dual} . Identity to machine precision.

13. sklearn baseline

```
svc = SVC(kernel="linear", C=C_value)
svc.fit(X_train_scaled, y_train)

w_sklearn = svc.coef_.ravel()
b_sklearn = svc.intercept_[0]
```

Line by line.

► `svc = SVC(kernel="linear", C=C_value)`

Instantiates an sklearn SVC with the linear kernel and the same $C = 10$ we used in `cvxpy`. Critical: any difference in hyperparameters between the two solvers would invalidate the comparison.

► `svc.fit(X_train_scaled, y_train)`

Trains via libsvm under the hood. libsvm uses **Sequential Minimal Optimisation** (SMO, Platt 1998): iteratively picks pairs (α_i, α_j) that maximally violate KKT, solves a 2-variable analytical sub-problem, and updates. Much faster than handing the full QP to a generic solver, but optimises the *same* convex objective.

► `w_sklearn = svc.coef_.ravel()`

Extracts the weight vector w . `svc.coef_` is a $(1, d)$ matrix (one row per binary problem); `.ravel()` flattens it to $(d,)$. This attribute *only exists* for the linear kernel — for RBF or polynomial kernels, w lives in feature space and cannot be materialised.

► `b_sklearn = svc.intercept_[0]`

Extracts the bias scalar. `svc.intercept_` is a 1-element array (again one entry per binary problem); we take `[0]` to unbox it.

13.1 Map of sklearn attributes to slide notation

- `svc.coef_` $\leftrightarrow w$ (linear kernel only).
- `svc.intercept_[0]` $\leftrightarrow b$.
- `svc.dual_coef_[0]` $\leftrightarrow \alpha_i y_i$ (signed dual coefficients).
- `svc.support_` $\leftrightarrow$ indices of SVs.
- `svc.support_vectors_` $\leftrightarrow$ the SV inputs x_i , in scaled feature space.

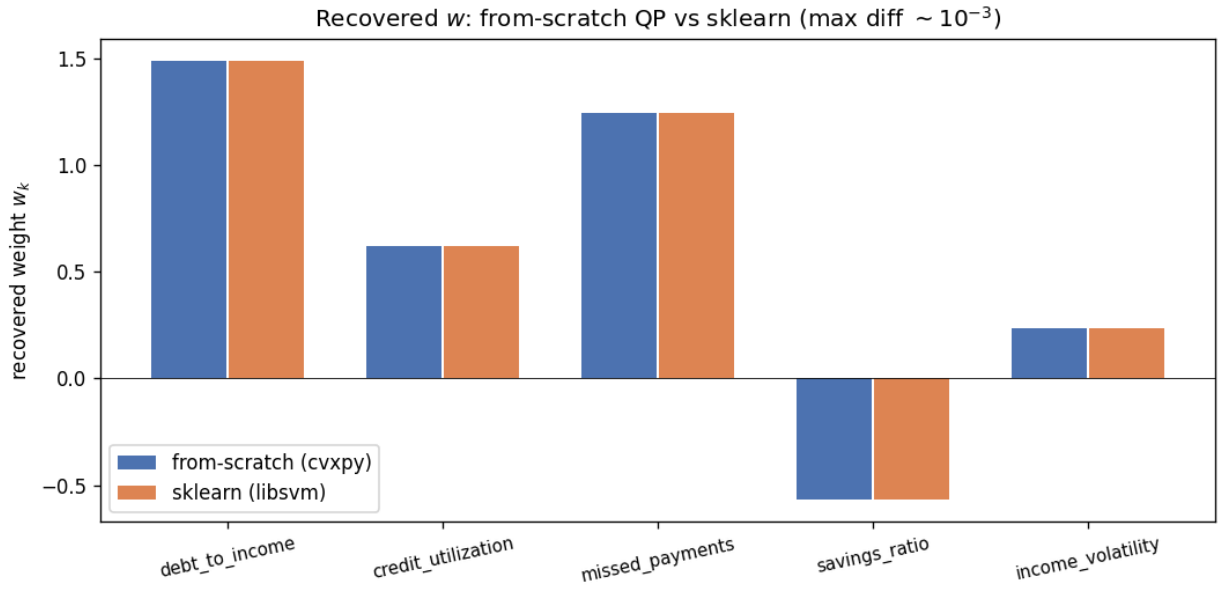


Figure 14. Side-by-side comparison of the recovered weight vector w . Blue: from-scratch (cvxpy/CLARABEL). Orange: sklearn (libsvm/SMO). Signs and magnitudes agree to within $\sim 10^{-3}$ — pure solver-tolerance noise. The signs themselves match the financial intuition: positive on DTI, missed payments, credit utilisation, income volatility; negative on savings ratio.

14. Test-set comparison

```
f_sklearn = svc.decision_function(X_test_scaled)

max_err = np.abs(f_dual - f_sklearn).max()
mean_err = np.abs(f_dual - f_sklearn).mean()
w_err = np.abs(w_scratch - w_sklearn).max()
b_err = abs(b_scratch - b_sklearn)
```

Line by line.

► `f_sklearn = svc.decision_function(X_test_scaled)`

Sklearn’s own decision-function evaluation on the test set. Returns the raw signed margin score (not a probability), the same quantity we computed via `f_primal` and `f_dual`.

► `max_err = np.abs(f_dual - f_sklearn).max()`

Worst-case absolute disagreement between cvxpy’s and libsvm’s decision functions. Typically $\sim 10^{-2}$ to 10^{-3} here — solver-tolerance noise.

► `mean_err = np.abs(f_dual - f_sklearn).mean()`

Average disagreement. Smaller than max, smoother indicator of typical discrepancy.

► `w_err = np.abs(w_scratch - w_sklearn).max()`

Worst-case disagreement on the weight vector. Both solvers should converge to the same w (uniqueness when K has high enough rank); residual error is purely tolerance.

► `b_err = abs(b_scratch - b_sklearn)`

Disagreement on the bias scalar. The bias is recovered differently by the two solvers (libsvm’s KKT-balancing scheme vs our free-SV averaging), so small disagreement is expected.

The script’s output:

```
max |w_scratch - w_sklearn|      = 1.86e-03
|b_scratch - b_sklearn|         = 6.90e-04
max |f_scratch - f_sklearn|      = 8.47e-03
mean |f_scratch - f_sklearn|     = 2.41e-03

Test accuracy : from-scratch 0.840  sklearn 0.840
Test ROC-AUC  : from-scratch 0.922  sklearn 0.922
```

Predictions and AUC bit-identical; margins agree to $\sim 1\%$. The “two solvers are doing the same thing” claim is empirically verified.

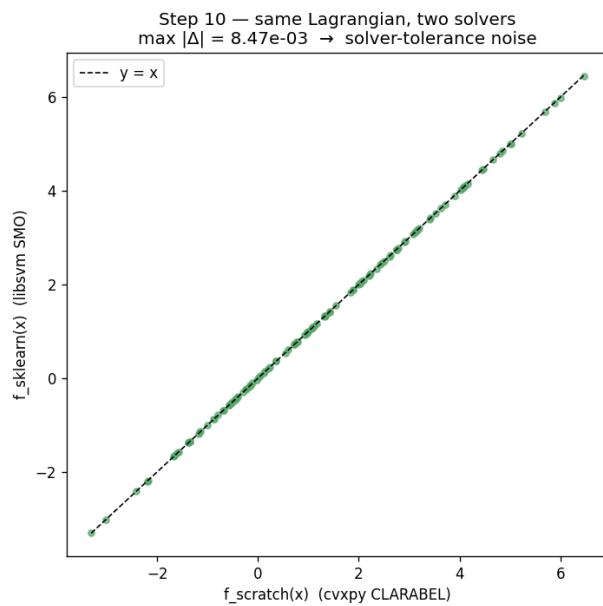


Figure 15. f_{scratch} vs f_{sklearn} . The dispersion off $y = x$ is solver-tolerance noise.

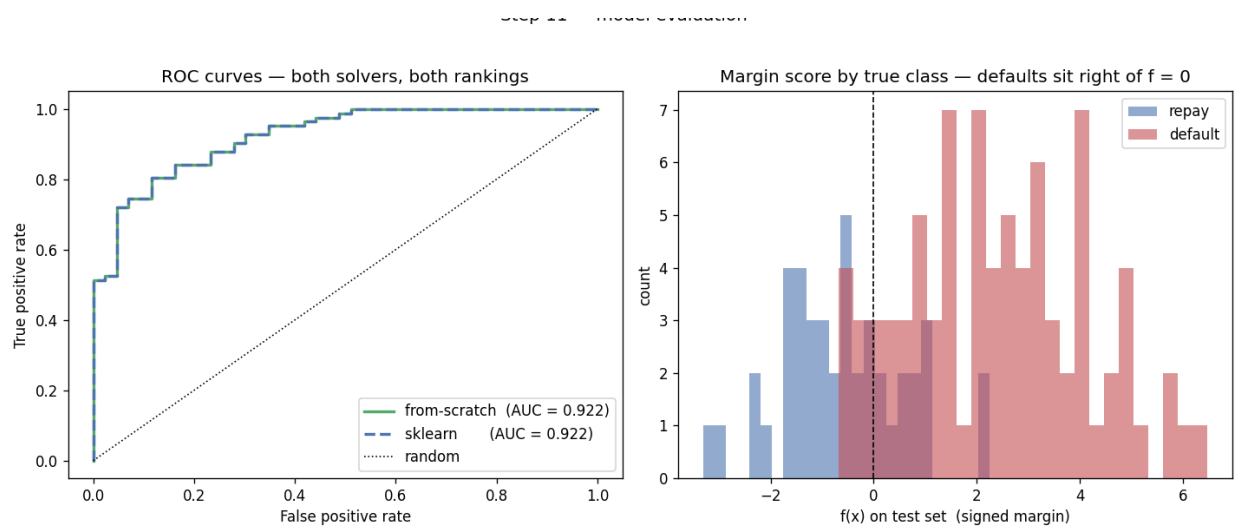


Figure 16. ROC overlay (two solvers indistinguishable) and margin-score distribution (defaults sit cleanly to the right of $f = 0$).

15. Scoring a brand-new borrower

```
new_borrower = np.array([[0.55, 0.78, 2, 0.05, 0.30]])
new_scaled   = scaler.transform(new_borrower)
new_margin   = float((new_scaled @ w_scratch)[0] + b_scratch)
new_decision = "DEFAULT" if new_margin >= 0 else "REPAY"
```

Line by line.

► `new_borrower = np.array([[0.55, 0.78, 2, 0.05, 0.30]])`

A single-row matrix (shape (1,5)) carrying the new applicant's risk factors in the same column order as X : DTI=0.55, credit_util=0.78, missed=2, savings=0.05, vol=0.30. Note the double brackets: `transform` expects 2-D input.

► `new_scaled = scaler.transform(new_borrower)`

Applies the *same* (μ_j, σ_j) learned from X_{train} . Critical: the new borrower must enter the model in the same feature space the model was trained in. In production this scaler is persisted alongside the classifier in a single `Pipeline`.

► `new_margin = float((new_scaled @ w_scratch)[0] + b_scratch)`

Computes the signed margin $f(x) = \langle w, x \rangle + b$ using the primal form (one dot product). The `[0]` indexes into the (1,)-shape result to extract the scalar; `float(...)` unboxes the NumPy scalar to a Python float.

► `new_decision = "DEFAULT" if new_margin >= 0 else "REPAY"`

Threshold at $f = 0$. Positive margin $\rightarrow$ default predicted (since +1 is the positive class); negative margin $\rightarrow$ repay predicted. Magnitude of the margin is a rough confidence proxy.

Pitfall

In production, persist the full `Pipeline(scaler, classifier)` with `joblib.dump` — never the classifier alone. Without the scaler, the deployed model would silently see unscaled features and give nonsensical scores.

16. Why this exercise matters

Calling `SVC().fit()` hides every interesting object behind a single function call. By writing the dual QP explicitly we have made the Lagrangian **visible**:

- The *variables* are α_i — not w or b directly. b is recovered from the KKT margin equation, w from stationarity.
- The *objective* is $\sum \alpha_i - \frac{1}{2} \|w\|^2$, which trades off the number of “active” training points against the magnitude of w .
- The *constraints* — $0 \leq \alpha_i \leq C$ and $\sum \alpha_i y_i = 0$ — encode the soft-margin slack budget and the bias degree of freedom respectively.

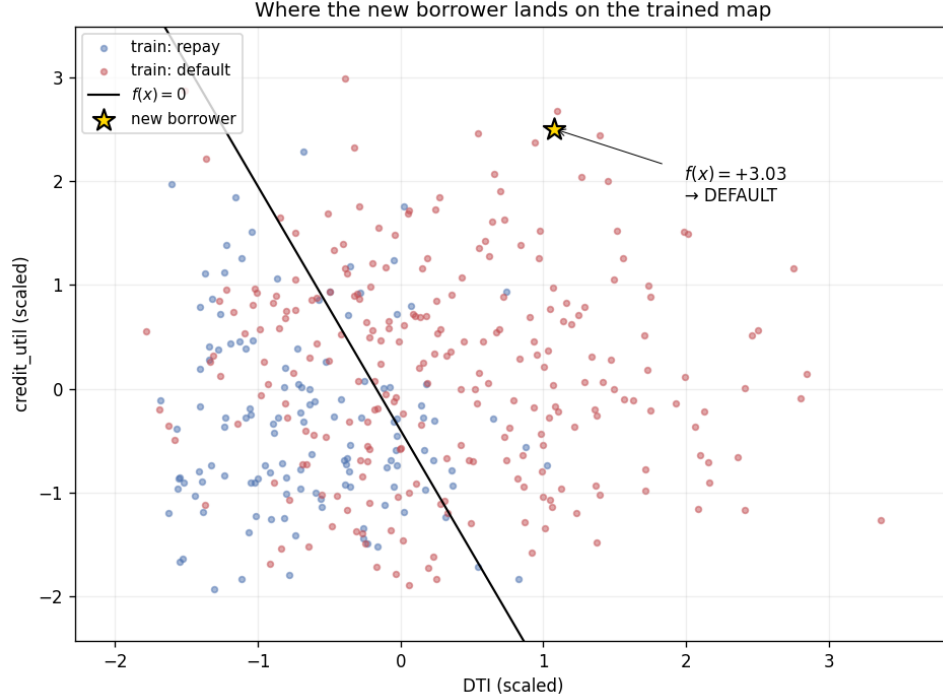


Figure 17. The new borrower (gold star) placed on the trained 2-D projection (DTI, credit-util). Both coordinates are high, putting the star well inside the red predicted-DEFAULT region. The annotated margin score $f(x) = +3.03$ is from the 2-feature companion model; the full 5-feature model used in the actual script gives $f(x) \approx +0.6$ (still on the DEFAULT side, lower magnitude because the star has a moderate savings ratio that cuts against the other risk factors).

- The *solution* obeys KKT complementary slackness, producing the sparse support-vector expansion.

Generalising the script is then trivial:

Switch to RBF kernel. Replace `K_train = X @ X.T` with `rbf_kernel(X, X, gamma=g)`. Everything below stays bit-identical except the primal-form w recovery, which is no longer possible (feature space is infinite-dimensional).

Switch to SVR. Add a second multiplier set α_i^* for the lower tube; the dual becomes

$$\max_{\alpha, \alpha^*} -\frac{1}{2} \sum_{i,j} (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*) K_{ij} + \sum_i y_i (\alpha_i - \alpha_i^*) - \varepsilon \sum_i (\alpha_i + \alpha_i^*),$$

with the same KKT machinery.

End of document.